

CVE: CVE-2023-24329

Description: An issue in the `urllib.parse` component of Python before 3.11.4 allows attackers to bypass blocklisting methods by supplying a URL that starts with blank characters.

Vendor: Python Software Foundation

Version: `=> 3.11.3`

7.5
CVSSV3

CVE-2023-24329

Published: 17/02/2023 Updated: 12/08/2023

CVSS v3 Base Score: 7.5 | Impact Score: 3.6 | Exploitability Score: 3.9

[Subscribe to Python](#)

Vulnerability Summary

An issue in the `urllib.parse` component of Python prior to 3.11.4 allows malicious users to bypass blocklisting methods by supplying a URL that starts with blank characters.

Background Story

Python is the most popular programming language in the world. The exploit of CVE-2023-24329 is pretty easy, but its impact is

significant because the `urllib.parse` library is used in many web applications to prevent blacklisting attacks, such as

injection and file upload. When this library is broken, attackers can manipulate inputs and bypass filters

the bug affects any version before `3.11.4`, I'm using `python3 3.11.3` version which is the last version before the patch

Reproducing the Vulnerability

`urllib.parse` breaks the URL (A *Uniform Resource Locator*) strings up in components such as the schema (file, FTP, HTTP, HTTP)

that's used in the URL so when parsing a URL with URL as the `example.com`

```
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr  4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import urllib
>>> from urllib.parse import urlparse
>>> urlparse("http://example.com")
ParseResult(scheme='http', netloc='example.com', path='', params='', query='', fragment='') 1
>>> urlparse(" http://example.com")
ParseResult(scheme='', netloc='', path=' http://example.com', params='', query='', fragment='')2
```

[1] in normal case `urlparse` function detects the schema of the provided URL and this is used to prevent LFI (file)

[2] if the user adds space to the function will focus to detect the scheme which considers the URL schema with space as below

Demo of the Exploitation scenario

I made a simple web app to Demonstrate the exploit of this CVE in case the library is used to prevent LFI (Local File

Inclusion) using the Flask library which easily can be installed by `pip3 install flask` with `urllib` with two files `app.py` and

`index.html`

- `app.py`

```
from flask import Flask, render_template, request
import urllib.request
import urllib.error
app = Flask(__name__)
def safeURLOpener(inputLink):
    block_schemes = ["file", "gopher", "expect", "php", "dict", "ftp", "glob", "data"]
    block_host = ["instagram.com", "youtube.com", "tiktok.com"]
    input_scheme = urllib.parse.urlparse(inputLink).scheme
    input_hostname = urllib.parse.urlparse(inputLink).hostname
    if input_scheme in block_schemes:
        return "Input scheme is forbidden"
    if input_hostname in block_host:
        return "Input hostname is forbidden"
    try:
        target = urllib.request.urlopen(inputLink)
        content = target.read().decode('utf-8')
        return content
    except urllib.error.URLError as e:
        return "Error opening URL: " + str(e)
@app.route('/', methods=['GET', 'POST'])
def index():
    content = ""
    error = None
    if request.method == 'POST':
        domain = request.form.get('domain')
        if domain:
            content = safeURLOpener(domain)
    return render_template('index.html', content=content, error=error)
if name == '__main__':
    app.run(debug=True)
```

- `Index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>Domain Content Viewer</title>
</head>
<body>
    <h1>Domain Content Viewer</h1>
    <form method="post">
        <label for="domain">Enter a domain:</label>
        <input type="text" name="domain" id="domain" value="{{ request.form['domain'] }}">
        <button type="submit">Submit</button>
```

```
</form>
{% if content %}
<h2>Content:</h2>
<pre>{{ content }}</pre>
{% endif %}
{% if error %}
<h2>Error:</h2>
<p>{{ error }}</p>
{% endif %}
</body>
</html>
```

run the app using `python3`

```
PS C:\Users\yosef\Desktop\code\Smiple_app> python3 .\app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 929-486-947
```

which running by default to `127.0.0.1:5000`

in case submitting a common payload of LFI `file:c:\WINDOWS\win.ini` Windows-based



← → ↻ 127.0.0.1:5000

Domain Content Viewer

Enter a domain:

Content:

Input scheme is **forbidden**

the application returned with `Input schema is forbidden` because `file` schema blocked by the blocklist

After adding a space in the URL, the application blocklisting failed to detect the URL `schema` and causes bypassing of the

blocklist

  127.0.0.1:5000

Domain Content Viewer

Enter a domain:

Content:

```
; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1
```

Setting the Debugging Environment

the vulnerable library: <https://github.com/python/cpython/tree/3.11/Lib/urllib>

```
from urllib.parse import urlparse

url_to_parse = " https://www.vicarius.io/"

output = urlparse(url_to_parse)

print(output)
```

then add the breakpoint in the line that uses the function to trigger the vulnerable function

```
trigger.py x
1 from urllib.parse import urlparse
2
3 url_to_parse = " https://www.vicarius.io/"
4
5 output = urlparse(url_to_parse)
6
7 print(output)
```

start the debugger by clicking on the debugger button in the tools bar

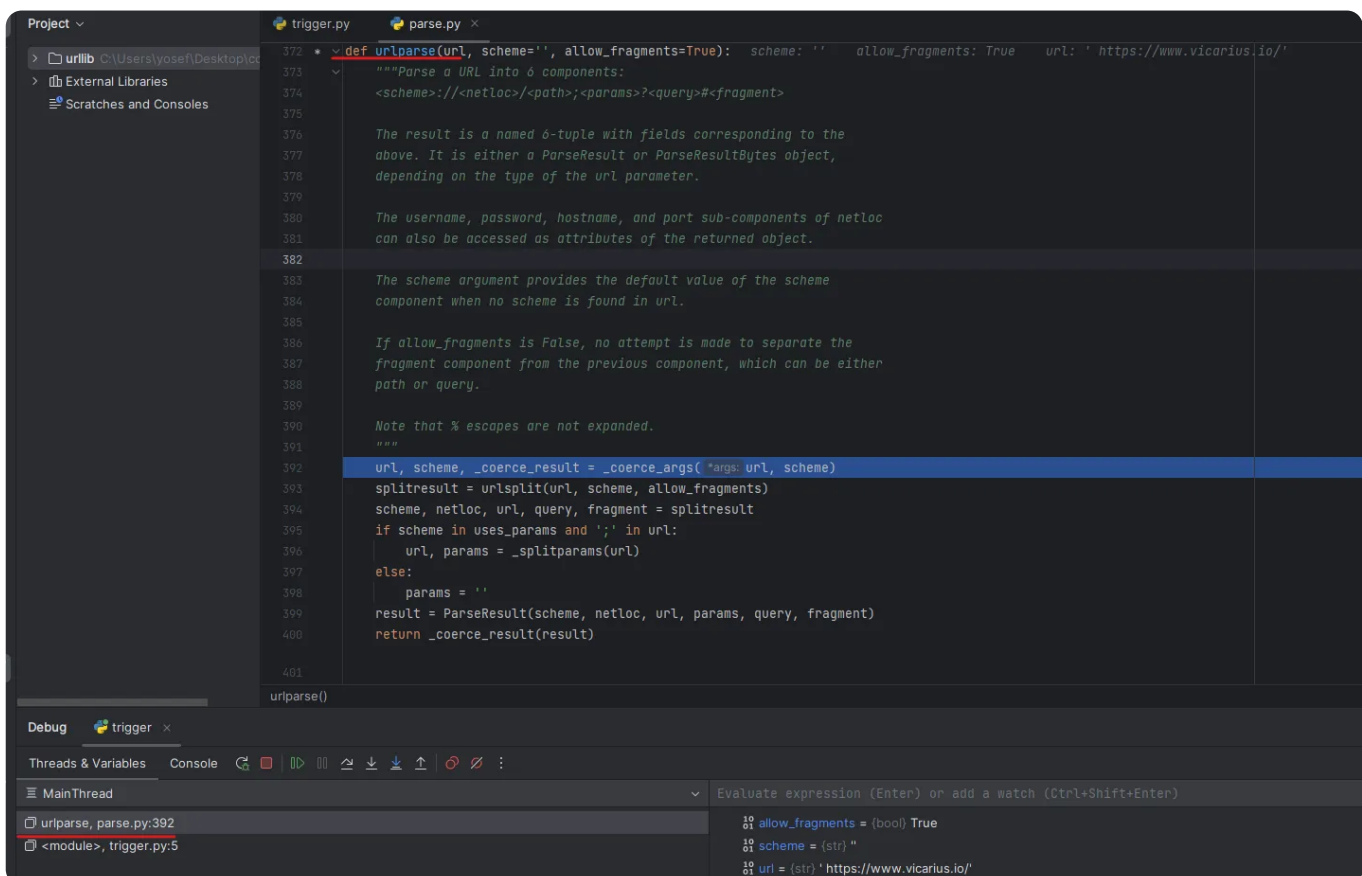


After running the debugger and stepping into it to let the IDE go to the source code of the installed Python version in the

machine, the default in Windows is `C:\Users\yosef\AppData\Local\Programs\Python\Python39\Lib\urllib`.

It appears that it calls the `urlparse` function, which exists in the `parse.py` file.

- `urlparse`



`urlparse` function first breaks down the provided argument into two pieces

the `scheme` and

`URL` set it to an empty string (`scheme=''`) as the default and the given

`URL`. The function then proceeds to parse the argument

by passing it to the `_coerce_args` function

`_coerce_args`

```
114 def _coerce_args(*args):  args: (' https://www.vicarius.io/', '')
115     # Invokes decode if necessary to create str args
116     # and returns the coerced inputs along with
117     # an appropriate result coercion function
118     # - noop for str inputs
119     # - encoding function otherwise
120     str_input = isinstance(args[0], str)
121     for arg in args[1:]:
122         # We special-case the empty string to support the
123         # "scheme='" default argument to some functions
124         if arg and isinstance(arg, str) != str_input:
125             raise TypeError("Cannot mix str and non-str arguments")
126     if str_input:
127         return args + (_noop,)
128     return _decode_args(args) + (_encode_result,)
```

`_coerce_args` function make sure the given argument if it was a string

return `noop` and if not raise an error `Cannot mix str and non-str arguments`

then `urlparse` call `urlsplit` to split the given url into `scheme`, `netloc`

(network location), `url`, `query`, and `fragment`.

`urlsplit`

```
url, scheme, _coerce_result = _coerce_args(*args: url, scheme)

for b in _UNSAFE_URL_BYTES_TO_REMOVE:  b: '\n'
    url = url.replace(b, "")
    scheme = scheme.replace(b, "")
```

`urlsplit` have all the important works, which use

`_UNSAFE_URL_BYTES_TO_REMOVE` variable as shown below to remove `'\t', '\r', '\n'` prevent injection

```
81 # Unsafe bytes to be removed per WHATWG spec
82 _UNSAFE_URL_BYTES_TO_REMOVE = ['\t', '\r', '\n']
83
```

Root Cause

```
i = url.find(':') i: 6
if i > 0:
    for c in url[:i]: c: ' '
        if c not in scheme_chars:
            break
    else:
        scheme, url = url[:i].lower(), url[i + 1:]
```

`for c in url[:i]` which checks for the characters in the URL first part before the colon (file schema) using `url.find(':')` for

all characters if `c` among the valid characters that exist in `scheme_chars` variable as shown below:

```
# Characters valid in scheme names
scheme_chars = ('abcdefghijklmnopqrstuvwxyz'
               'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
               '0123456789'
               '+-.')
```

if `c` characters existed in `scheme_chars` the code will proceed and save it as the `scheme` component and when and the root cause of this CVE when the code fails to get the the `schema` the code processing without getting the schema as shown in the stack tracing below:

```
✓ v = {SplitResult: 5} SplitResult(scheme="", netloc="", path='https://www.vicarius.io/', query="", fragment=")
10 01 fragment = {str} ""
10 01 hostname = {NoneType} None
10 01 netloc = {str} ""
10 01 password = {NoneType} None
10 01 path = {str} 'https://www.vicarius.io/'
10 01 port = {NoneType} None
10 01 query = {str} ""
10 01 scheme = {str} ""
10 01 username = {NoneType} None
10 01 0 = {str} ""
10 01 1 = {str} ""
10 01 2 = {str} 'https://www.vicarius.io/'
10 01 3 = {str} ""
10 01 4 = {str} ""
10 01 __len__ = {int} 5
```

```

if url[:2] == '//':
    netloc, url = _splitnetloc(url, start: 2)
    if (('[' in netloc and ']' not in netloc) or
        (']' in netloc and '[' not in netloc)):
        raise ValueError("Invalid IPv6 URL")
if allow_fragments and '#' in url:
    url, fragment = url.split('#', 1)
if '?' in url:
    url, query = url.split('?', 1)
_checknetloc(netloc)
v = SplitResult(scheme, netloc, url, query, fragment)
_parse_cache[key] = v
return _coerce_result(v)

```

and then checks for the `netloc` which starts in line number 380 in the code checking the part before `//` and checks if `[]`

not exist in IPv6 URL format rise an error says `Invalid IPv6 URL` then it checks for the fragments (sections indicated by `#`)

and then the query and pass the `netloc` to `_checknetloc()` function

and for `_checknetloc(netloc)`

```

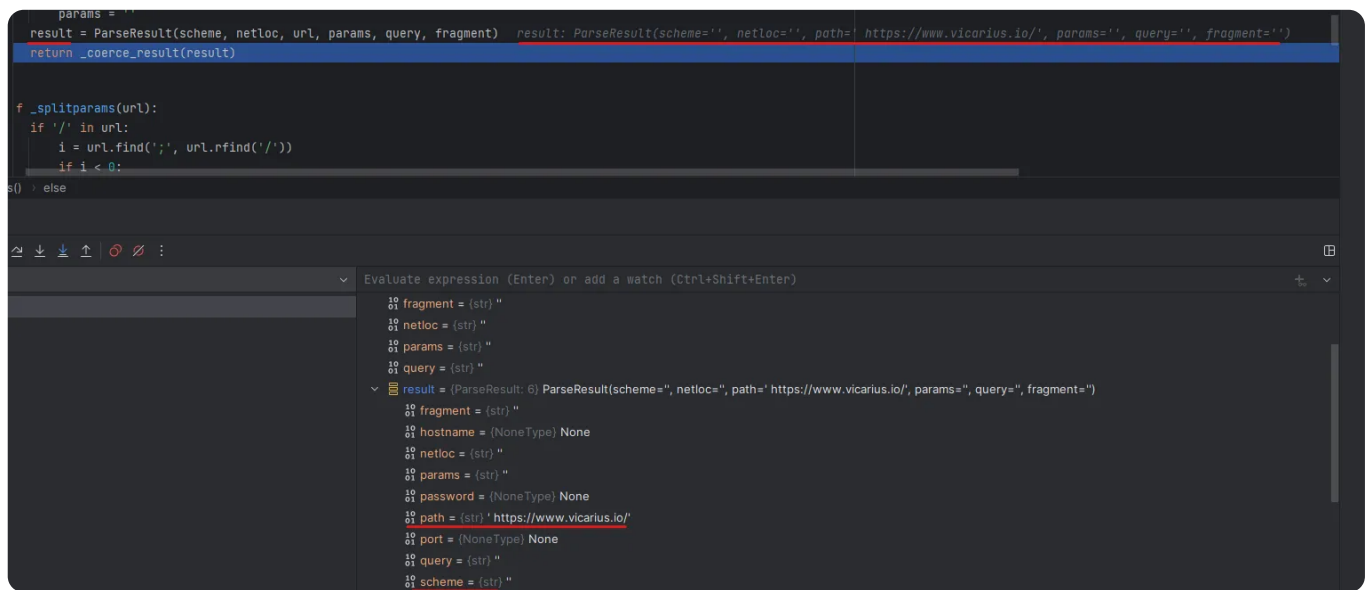
def _checknetloc(netloc):
    netloc: ''
    if not netloc or netloc.isascii():
        return
    # looking for characters like \u2100 that expand to 'a/c'
    # IDNA uses NFKC equivalence, so normalize for this check
    import unicodedata
    n = netloc.replace('@', '') # ignore characters already included
    n = n.replace(':', '') # but not the surrounding text
    n = n.replace('#', '')
    n = n.replace('?', '')
    netloc2 = unicodedata.normalize(_form: 'NFKC', n)
    if n == netloc2:
        return
    for c in '/?#@:':
        if c in netloc2:
            raise ValueError("netloc '" + netloc + "' contains invalid " +
                              "characters under NFKC normalization")

```

The `_checknetloc()` function handles the network location by first confirming whether it consists of ASCII characters or not. It then proceeds to replace symbols like '@', ':', '#', and '?' and normalizes the resulting string. Afterward, the function checks if these characters still exist; if they do, an error is raised

and then it saves the into variable `parseResult` which appears below picture of the code processed without getting the

`schema` and `netloc` components and save the URL into `path`



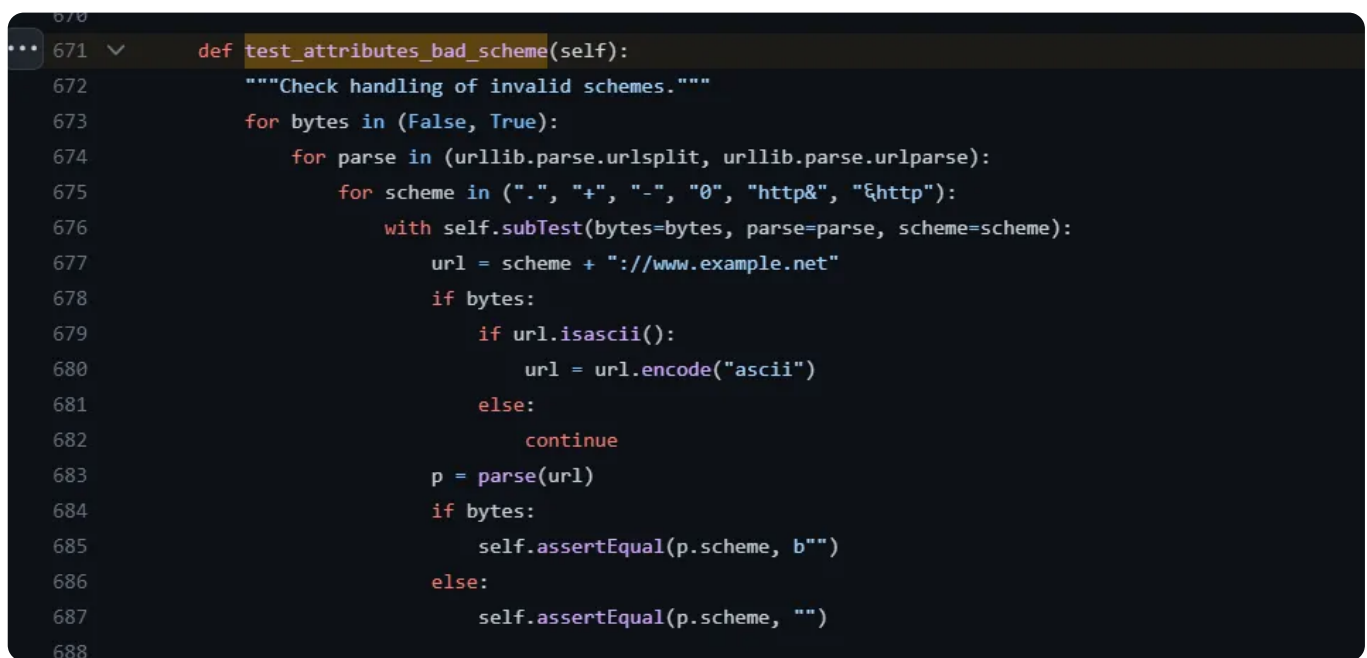
The screenshot shows a code editor with a Python function `_coerce_result` and its evaluation results. The function takes `url` as input and returns a `ParseResult` object. The evaluation results show the following values:

```
fragment = (str) ""
netloc = (str) ""
params = (str) ""
query = (str) ""
result = (ParseResult: 6) ParseResult(scheme="", netloc="", path='https://www.vicarius.io/', params="", query="", fragment="")
fragment = (str) ""
hostname = (NoneType) None
netloc = (str) ""
params = (str) ""
password = (NoneType) None
path = (str) 'https://www.vicarius.io/'
port = (NoneType) None
query = (str) ""
scheme = (str) ""
```

Patch diffing

link: <https://github.com/python/cpython/pull/99421/commits/a284d69de1d1a42714576d4a9562145a94e62127>

the modification of the library was by adding



The screenshot shows a code editor with a Python function definition `test_attributes_bad_scheme`. The function is defined as follows:

```
def test_attributes_bad_scheme(self):
    """Check handling of invalid schemes."""
    for bytes in (False, True):
        for parse in (urllib.parse.urlsplit, urllib.parse.urlparse):
            for scheme in (".", "+", "-", "0", "http&", "&http"):
                with self.subTest(bytes=bytes, parse=parse, scheme=scheme):
                    url = scheme + "://www.example.net"
                    if bytes:
                        if url.isascii():
                            url = url.encode("ascii")
                        else:
                            continue
                    p = parse(url)
                    if bytes:
                        self.assertEqual(p.scheme, b"")
                    else:
                        self.assertEqual(p.scheme, "")
```

`test_attributes_bad_scheme` function which checks by looping a range of possible invalid

scenarios schemes, including `(".", "+", "-", "0", "http&", "&http")` and non-ascii to prevent the bypassing using a blank as the previous

exploitation

Mitigation

Upgrade Python to the lastest version [3.11.4](#)

Final Thoughts

after diving deep into the source code of the Python library [urllib](#) and debugging the library we take a look can simple bugs

do a significant impact as we see simple plank can bypass any blacklisting

Reference:

<https://github.com/python/cpython/issues/102153>